

Deep Learning

1. Introduction

Romain Tavenard

Course details

- 24 hours
- Instructor: Romain Tavenard romain.tavenard@univ-rennes2.fr
- Tools:
 - Datalab (JupyterHub) at <https://datalab.univ-rennes.fr>
 - or Jupyter Notebooks running on your machine
(need torch installed)
- Evaluation (equal weights)
 - Midterm paper exam
 - Research paper reading
 - Lab session

Pre-requisites

- Basics of Python coding
- A (tiny) bit of calculus
 - What's the derivative of a function?
 - Functions of several variables
- Machine Learning topics
 - Empirical risk optimization and its limitations
 - Model evaluation & selection (cross-validation)

Our first model: the Perceptron

- Input: $x = \{x_0, \dots, x_D\}$
- Output: a
- Parameters to be optimized: $\{w_0, \dots, w_D, b\}$
- Activation function (chosen a priori): φ

$$a = \varphi \left(\sum_j w_j x_j + b \right)$$

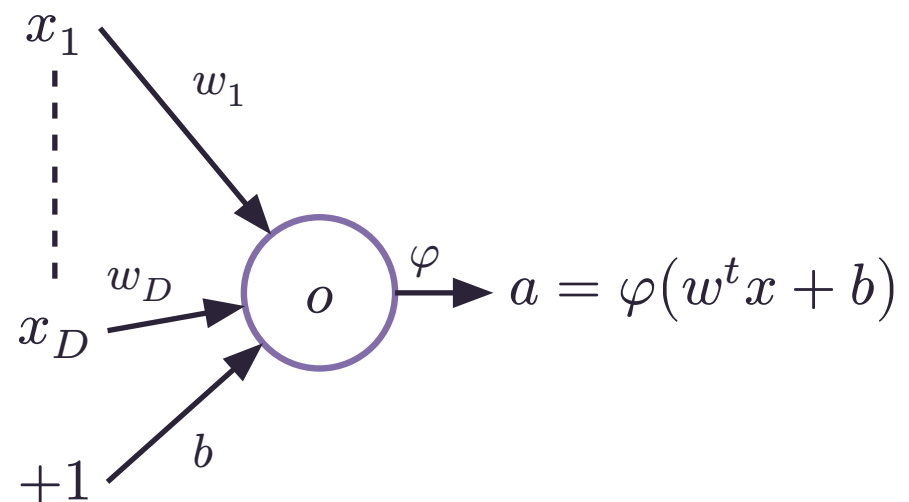


Figure: Perceptron diagram

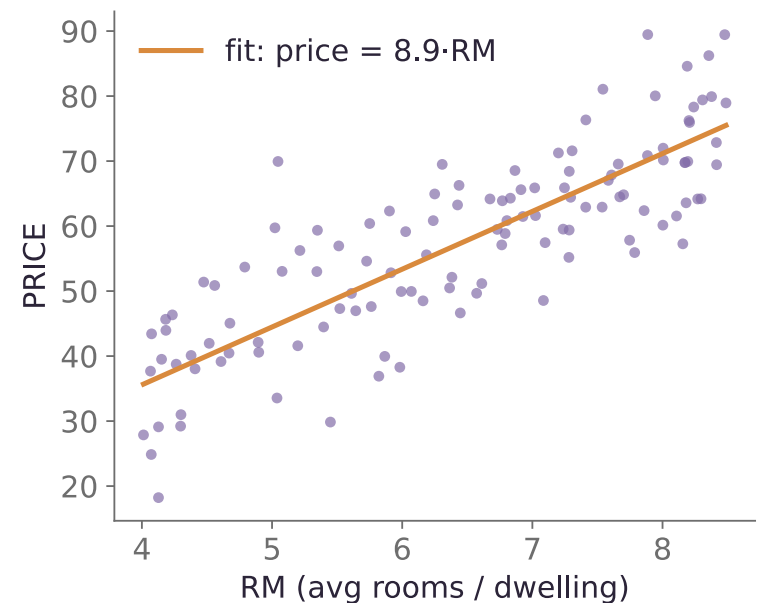
Motivating example

- Dataset: Boston housing prices
- Toy task: predict housing price (PRICE) based on average number of rooms per dwelling (RM)
- Chosen model: linear regression without intercept

$$\text{PREDICTED PRICE} = w_0 \times \text{RM}$$

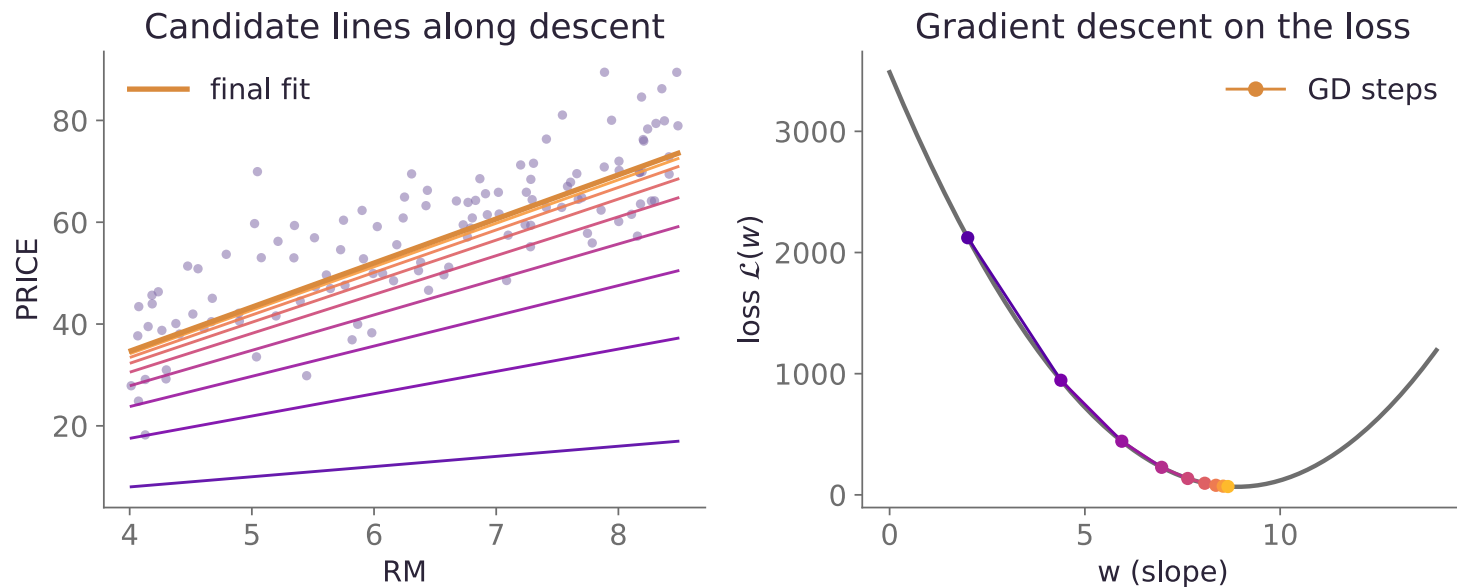
- Cost function (also called loss):
Mean Squared Error

$$\frac{1}{n} \sum_{i=1}^n (\text{PREDICTED PRICE}_i - \text{PRICE}_i)^2$$



Boston housing: RM vs PRICE

Motivating example: Optimization



Candidate fits and gradient-descent steps on the loss

- Goal: Find w_0 that minimizes our loss
- Testing many values at random is not tractable
- Alternative strategy: Gradient descent

$$w^{(t+1)} \leftarrow w^{(t)} - \rho \nabla_w \mathcal{L}(w^{(t)})$$

Course outline

- Model architectures
 - Multi-Layer Perceptrons (for tabular data)
 - Convolutional models (for images)
- Different losses for different learning tasks
- Optimization strategies (variants of gradient descent)